AI reliability platform

A gateway that keeps answering when the AI provider goes down.

180 of 180 requests succeeded while the primary provider was down.

Median latency rose from about 140 ms to about 290 ms on the fallback.

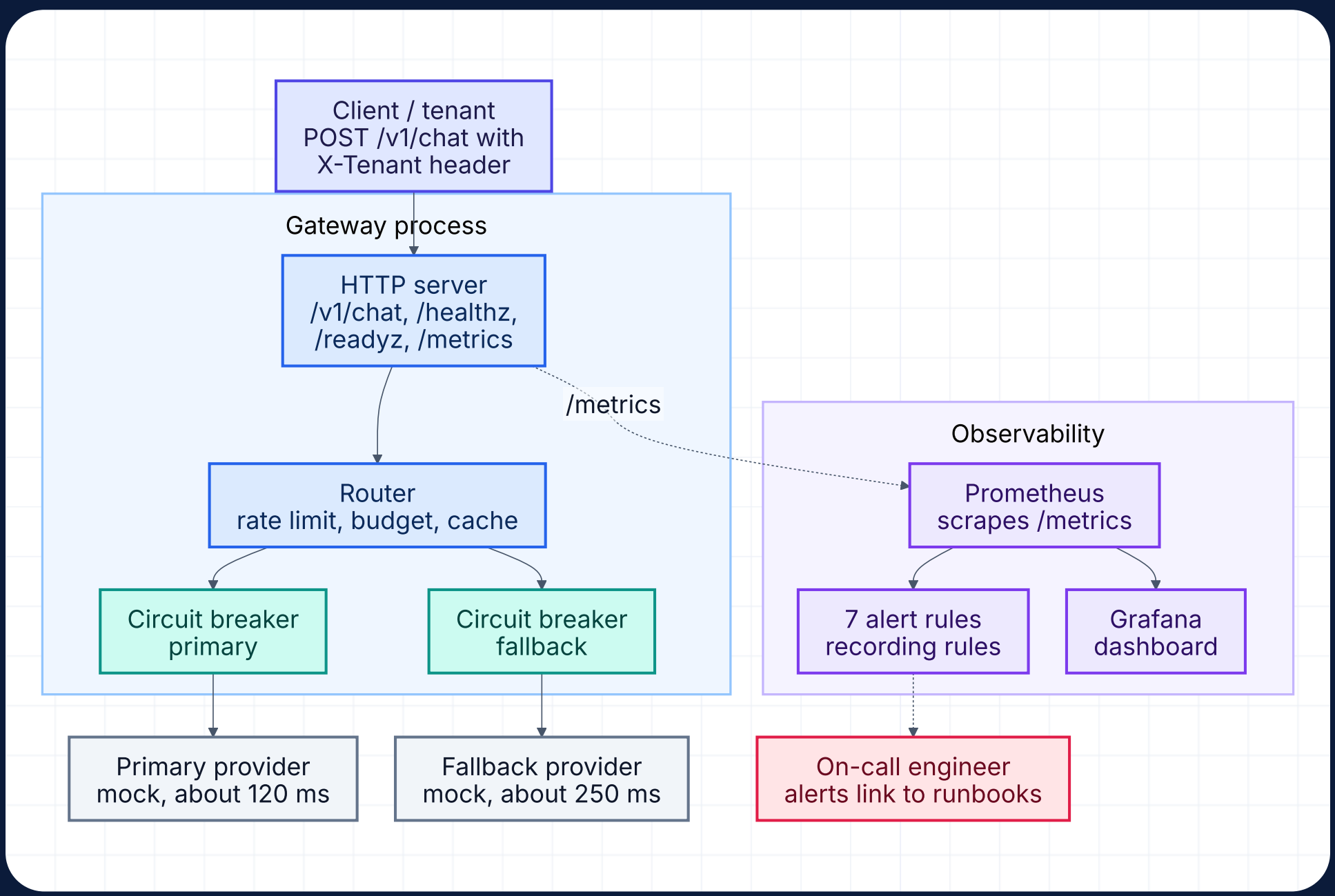
The circuit breaker opened, then closed by itself once the provider recovered.

Jaime Castillo Araujo

5 schematics follow. Code: github.com/Joxejaime/projects-2026

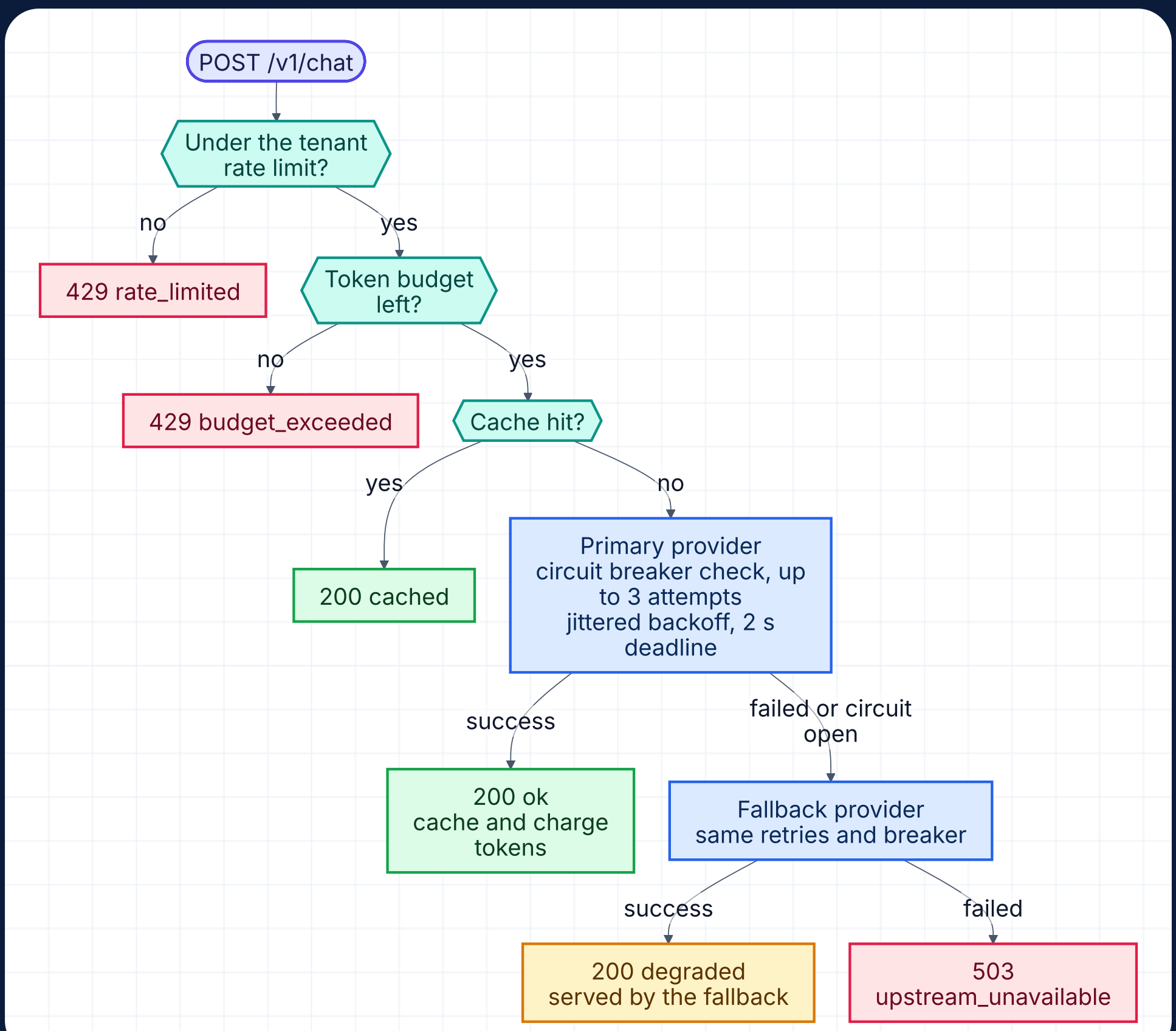
An AI gateway built to keep answering when a provider fails

Rate limits, budgets, cache, two providers, metrics and SLO alerts in one small service.



Every request takes one of six paths, and each one is measured

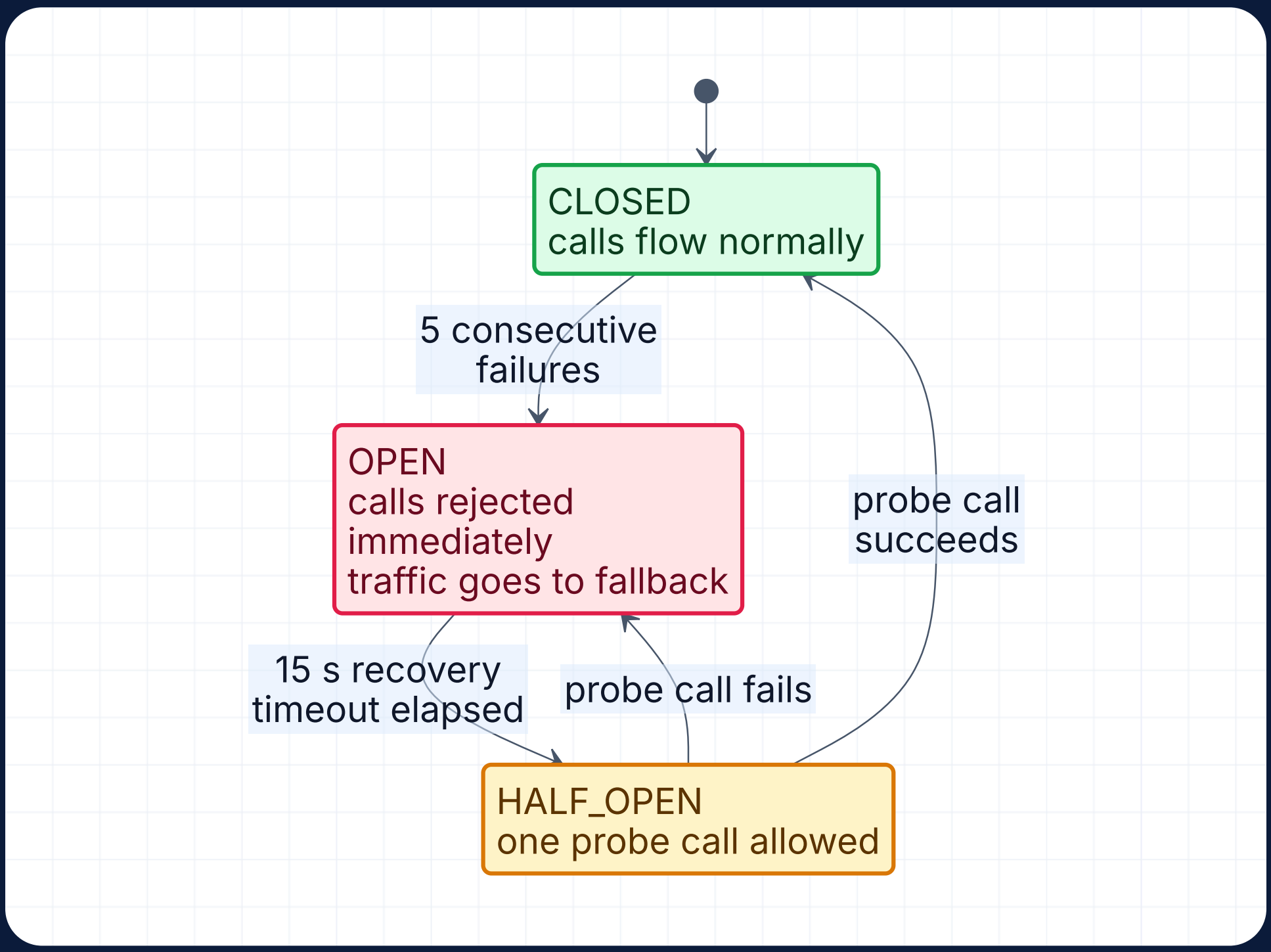
Limits first, then cache, then the primary provider, then the fallback.



Request · Check · Provider call · Success · Fallback success · Rejected

A circuit breaker stops hammering a provider that is down

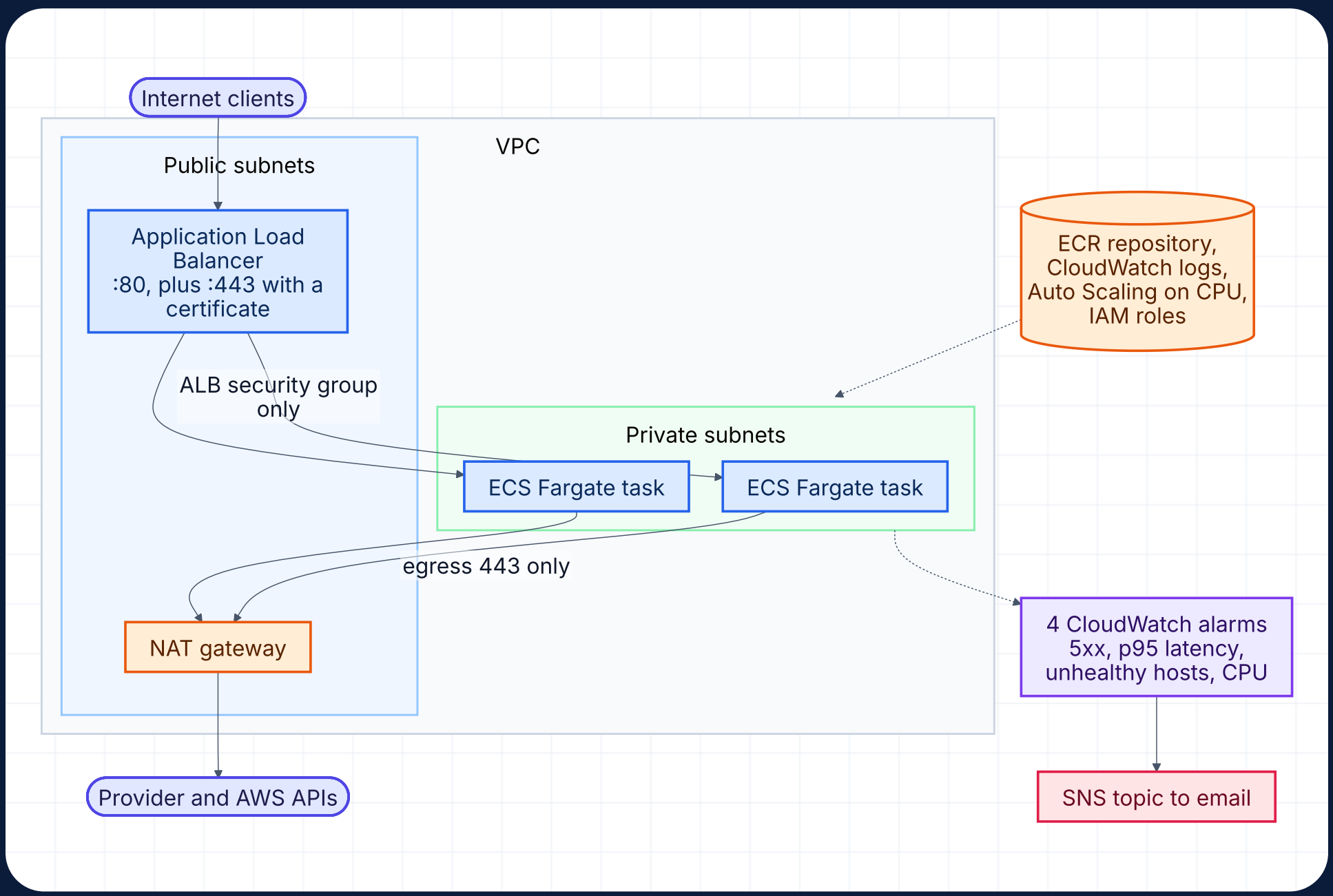
Five failures open it. After 15 seconds one probe call decides whether to close it.



Calls flow Calls rejected One probe allowed

The AWS deployment: private tasks behind a load balancer

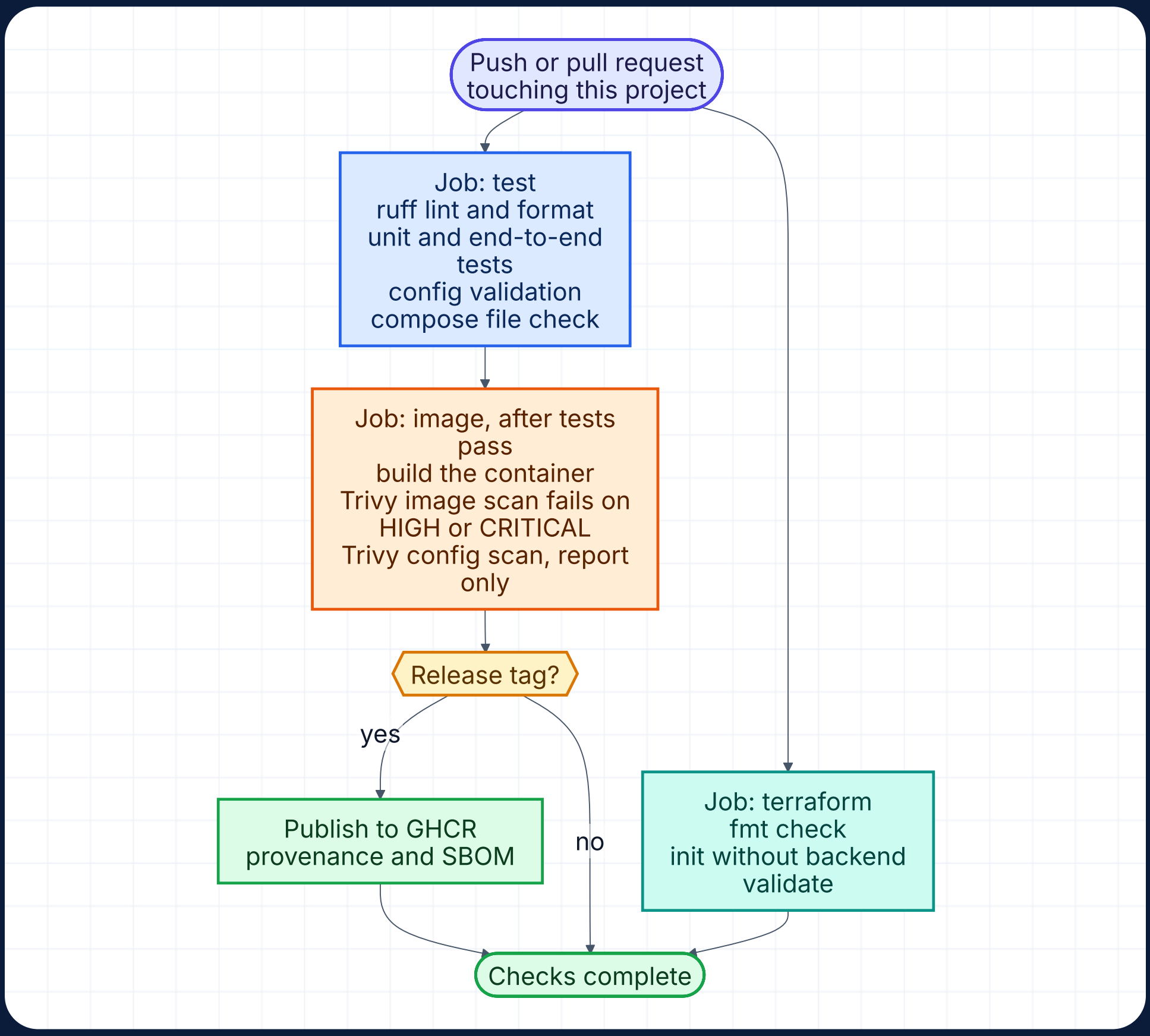
ECS Fargate, ALB, NAT, autoscaling and four CloudWatch alarms, all in Terraform (not yet applied).



Internet · Load balancer and tasks · AWS service · Monitoring · Notification

Every change is linted, tested, scanned and validated

Tests, Terraform checks and a Trivy image scan run before anything is released.



What I ran, and what I did not

Run and passing

- 40 unit and end-to-end tests
- A live outage drill with recorded results
- Alert rules, dashboard and Kubernetes manifests checked

Written, not yet run

- Docker image build
- Terraform validate and apply
- Prometheus firing the alerts live

github.com/Joxejaime/projects-2026

projects/ai-reliability-platform, with tests, runbooks and these schematics